

SpeakATX: Technical Report

Overview

[SpeakATX](#) is a website made using React, Vite, Bootstrap, and Node.js as a frontend, and Python with a MySQL server for the backend. Its goal is to provide resources to minimal English and non-English speakers in Austin, TX.

- The splash page contains a brief overview of SpeakATX's purpose, a chart of the languages we support, and some basic navigation.
 - This page is linked in our navigation bar at the top of the site.
- The about page reiterates SpeakATX's purpose and provides information about the developers.
 - This page is linked in our navigation bar at the top of the site.
- There is one page each for services, language communities, and job postings.
 - Each of these pages are linked in our navigation bar at the top of the site.
- There are additional pages for each individual instance of services, communities, and job postings.
 - These are linked as cards in their respective parent pages.
- The search page contains a search bar that allows users to search across instances of all models
 - This page is linked in our navigation bar

About Page

The about page reiterates SpeakATX's purpose and provides information about the developers. For each developer this includes their name, a picture, and a short bio.

The about page also tracks git commits and issues for each developer. This is updated dynamically by fetching data from GitLab's API.

- For both commits and issues, GET requests are made per page of 100 items, looping through pages until there is no data left.
- The resulting commits/issues are then mapped to their authors/assignees and counted.

The about page has a link to the API documentation at the very bottom.

Models, Instances, and Attributes

SpeakATX has 3 core models: services, communities, and job postings. Each model contains some number of instances. Each instance has several attributes.

Each model page displays the number of instances and a card for each instance. Each card contains an image and lists the instance attributes. These cards serve as clickable links which lead to their individual instance pages.

Instance pages for each model contain further information and a button that links to the external website for that instance. Varying by model, some instances may additionally feature images and maps. We have added a back button to each instance page for convenience, as we expect users to browse and click through multiple pages.

Instances also connect to related instances of other models. Currently, SpeakATX will display cards at the bottom of the page for related instances that match the languages of the current instance.

- Language is an important attribute in SpeakATX. All instances have at least one language.
- Instances can have multiple languages.

Related instances are found by iterating through the lists of instances that belong to the other models and selecting those that have at least one matching language.

We store attribute information for all instances of all models in the `instances.jsx` file as static arrays belonging to the `Instances` class. Currently, they have been manually inputted. For the next phase, they will be dynamically generated based on data fetched from our data sources. The attributes are as follows:

- Service attributes:
 - Name (string)
 - Rating (int 1-5)
 - Area of Austin (string)
 - Price (int)
 - Languages (string)
 - Map Location (string, parsed and used for map on frontend)
 - Description (string)
- Job attributes:
 - Name (string)
 - Title (string)
 - Hourly Pay (int)
 - Area of Austin (string)
 - Languages (string)
 - Description (string)
- Community attributes:
 - Name (string)
 - Area of Austin (string)
 - Community type (string)
 - Number of members (int)
 - Languages (string)
 - Description (string)

Searching, Sorting, and Filtering

We implement searching in two main locations. A search bar is present on each model page to allow for searching by keywords and phrases within that model. There is additionally a search that can be used to search instances of all models at once. This is implemented on the frontend by checking for matches in different fields of the instances, such as the title and description.

We weighed search relevance by assigning scores to each instance, where we add 10 points for an exact phrase match anywhere in the instance (including descriptions), 1 point for single word matches, and 3 points for each multi-word match. Title matches get an extra weight of 5 points. We chose to not consider service rating, community member count, or job pay since users are unlikely to search for these exact values. Search matches are highlighted on all model pages and the search page.

Each model page contains a dropdown for sorting and a dropdown for filtering. Users can filter their results by language, one language at a time. At the same time, users can sort by any of the other attributes of the model, in either ascending or descending order. This is implemented on the frontend by simply sorting or filtering the lists of instances retrieved from our API based on the parameters required.

User Stories

Phase I:

We implemented four of our customer's user stories as requested, which were:

- Adding a translation feature on the splash page
- Adding a pie chart that displays the breakdown of foreign languages spoken in Austin
- Displaying the price of services on the cards instead of the budget range
- Fixing the layout on the About page so that the bios do not overlap with the team members' photos

We negotiated a user story that suggested we put services directly on the splash page because we felt that it wasn't fitting to put instances of a model on it. However, we did change the splash page's layout slightly since our customer felt that it looked empty.

Phase II:

We implemented all 5 of our customer's user stories, which were minor bug fixes.

- Fixing the coverage picture that wasn't displaying, and converted it to dynamically update
- Fix the pricing for the services
- Fix the back button since it clips
- Fill in the type of community
- Capitalize the languages on the community pages

Phase III:

We implemented all 5 of our customer's user stories, which mostly fixes and quality of life improvements.

- When doing searches on a model, the number of instances displayed doesn't change
- Hide API keys to prevent misuse
- When you press back, make it go back to the original page number the user was on
- Fix the Communities with Slashes in their Name not Loading
- Let user customize how many cells they see on a page

Hosting

[SpeakATX](#) is hosted on AWS Amplify. The domain name is registered on Namecheap. Our [API](#) is added as a record on Route53, and is running on an EC2 instance. Our database is a MySQL server running on Amazon RDS.

Data

To get the services, we used the Yelp API to query for services for different languages.

To get the jobs, we used SerpAPI to query Google Jobs for non-english and bilingual jobs.

To get the communities, we webscraped <https://www.meetup.com/> using Chromedriver, Selenium, and BeautifulSoup4.

On the frontend, we additionally use the Google Maps API on service instance pages to display a responsive map of the service location. This is not stored.

Each model contains various columns relevant to the model, such as name, language, or price, as well as an ID column serving as the primary key. Each instance within a model has a unique ID.

We hosted our data with Amazon RDS. It has a database called SpeakATX with three tables: Services, Communities, and Jobs.

To populate the database, we process the data we retrieve from APIs and scraping, and use sqlalchemy to insert it into the database.

API

SpeakATX has a custom RESTful API.

- Documentation: <https://documenter.getpostman.com/view/42364456/2sAYXFiHS8#intro>

Our RESTful API has the following queries for each model:

- Get - returns a number of instances for the model. Paginated, with a default of 10 per request
- Get By ID - returns a specific instance given its ID

The frontend makes Get requests to the API when the website loads and stores the items it gets in the Instances class. This can then be used by the pages of the website that require instance info.

Docker

We have a frontend docker container for development, as well as a backend docker container that runs our endpoint application.

Frontend: `docker pull maya393/frontend:latest`

Backend docker `pull maya393/backend:latest`

Tests

We made frontend unit tests using Jest. These ensure that various functions used throughout the frontend work correctly.

We made acceptance tests using Selenium that test if the text on the front page is correct, if there is at least one instance for each model, if all 5 team members' cards are on the about page, if links on the navigation bar are working, and if the instance cards link to the correct instance page.

The backend endpoint unit tests use pytest. They make requests to our API and verify that the correct information is returned, such as the status code and fields.

The postman tests consist of checking if the status code is 200, the response is in JSON format, and all the fields are as expected.

Challenges

Phase I

In developing this portion of SpeakATX, we ran into some minor challenges. Most were simply due to the learning curve, as several of us were new to web development. Some examples include properly centering UI elements and getting images to load correctly. These challenges were overcome through reading documentation, browsing internet forums, and consulting LLMs such as ChatGPT.

Phase II

Getting our data proved to be a big challenge. The first step was finding sources of data that had complete information that could also be scraped or had an API. This was difficult as many sites forbid scraping, or didn't have APIs. After this, we had to parse the data we got from these sources to fit what we needed for our models. This also came with some challenges, with some instances not having crucial data like language or address, and we had to fill in the gaps using other bits and pieces of data.

Phase III

Our main challenge this phase was cleaning our pages up and getting all the new features we added, such as sorting, pagination, and searching, to work nicely together. This was mainly challenging due to how each feature was created independently, and some effort was required to ensure they didn't conflict.